

JavaScript String Methods

Complete Reference Guide with Examples & Interview Q&A

[Syntax](#) • [Examples](#) • [Input/Output](#) • [Interview Questions](#)

Table of Contents

- 1. Introduction
- 2. charAt()
- 3. charCodeAt()
- 4. codePointAt()
- 5. at()
- 6. concat()
- 7. includes()
- 8. startsWith()
- 9. endsWith()
- 10. indexOf()
- 11. lastIndexOf()
- 12. search()
- 13. match()
- 14. matchAll()
- 15. slice()
- 16. substring()
- 17. substr() (legacy)
- 18. split()
- 19. replace()
- 20. replaceAll()
- 21. toUpperCase() / toLowerCase()
- 22. trim() / trimStart() / trimEnd()
- 23. padStart() / padEnd()
- 24. repeat()
- 25. normalize()
- 26. localeCompare()
- 27. toString() / valueOf()
- 28. String.fromCharCode() / fromCodePoint()
- 29. String.raw()
- 30. Interview Questions (Q / Input / Output)

1. Introduction

Strings in JavaScript are immutable primitive values. Every "string method" returns a **new** string (or array/value) rather than modifying the original string in place. Strings can be created using single quotes, double quotes, backticks (template literals), or the `String()` constructor. Below is a complete, example-driven walkthrough of every commonly used built-in String method, followed by a set of interview-style practice questions with input and output.

```
let s1 = 'Hello';
let s2 = "World";
let s3 = `Hello ${s2}`; // template literal
console.log(s3);        // Hello World
```

2. Character Access Methods

1 charAt(index)

Returns the character at the specified index. Returns an empty string if the index is out of range.

```
str.charAt(index)
```

```
let str = "JavaScript";
console.log(str.charAt(0)); // input index: 0
console.log(str.charAt(4));
console.log(str.charAt(20)); // out of range
```

Output: J S (empty string)

2 charCodeAt(index)

Returns the UTF-16 code unit (a number) at the given index.

```
str.charCodeAt(index)
```

```
let str = "ABC";
console.log(str.charCodeAt(0));
console.log(str.charCodeAt(1));
```

Output: 65 66

3 codePointAt(index)

Similar to charCodeAt, but works correctly with Unicode characters outside the Basic Multilingual Plane (e.g. emojis).

```
str.codePointAt(index)
```

```
let heart = "♥";
console.log(heart.codePointAt(0));
let str = "A";
console.log(str.codePointAt(0));
```

Output: 10084 65

4 at(index)

Returns the character at the given index. Unlike charAt, it accepts negative indices (counting from the end).

```
str.at(index)
```

```
let str = "Hello";
console.log(str.at(0));
console.log(str.at(-1)); // last character
```

Output: H o

3. Combining & Searching Methods

5 concat(str2, str3, ...)

Joins two or more strings and returns a new string.

```
str.concat(str2, str3, ...)
```

```
let a = "Hello";
let b = "World";
console.log(a.concat(" ", b, "!"));
```

Output: Hello World!

6 includes(searchValue, start)

Checks whether a string contains a given substring. Returns true/false.

```
str.includes(searchValue, start)
```

```
let str = "Learning JavaScript is fun";
console.log(str.includes("JavaScript"));
console.log(str.includes("Python"));
```

Output: true false

7 startsWith(searchValue, start)

Checks whether a string starts with the given substring.

```
str.startsWith(searchValue, start)
```

```
let str = "Hello World";
console.log(str.startsWith("Hello"));
console.log(str.startsWith("World", 6));
```

Output: true true

8 endsWith(searchValue, length)

Checks whether a string ends with the given substring.

```
str.endsWith(searchValue, length)
```

```
let str = "Hello World";
console.log(str.endsWith("World"));
console.log(str.endsWith("Hello", 5));
```

Output: true true

9 indexOf(searchValue, start)

Returns the index of the first occurrence of a substring, or -1 if not found.

```
str.indexOf(searchValue, start)
```

```
let str = "Hello World, Hello JS";
console.log(str.indexOf("Hello"));
console.log(str.indexOf("Hello", 5));
console.log(str.indexOf("Python"));
```

Output: 0 13 -1

10 lastIndexOf(searchValue, start)

Returns the index of the last occurrence of a substring, searching backwards.

```
str.lastIndexOf(searchValue, start)
```

```
let str = "Hello World, Hello JS";
console.log(str.lastIndexOf("Hello"));
```

Output: 13

11 search(regexp)

Searches for a match using a regular expression and returns the index, or -1.

```
str.search(regex)
```

```
let str = "The rain in Spain";  
console.log(str.search(/ain/));  
console.log(str.search(/xyz/));
```

Output: 5 -1

12 match(regex)

Retrieves the matches of a string against a regular expression, returned as an array.

```
str.match(regex)
```

```
let str = "Price: $10, $20, $30";  
console.log(str.match(/\$\d+/g));
```

Output: ['\$10', '\$20', '\$30']

13 matchAll(regex)

Returns an iterator of all matching results, including capture groups (regex must be global).

```
str.matchAll(regex)
```

```
let str = "cat bat hat";  
let matches = [...str.matchAll(/(\w)at/g)];  
matches.forEach(m => console.log(m[0], m[1]));
```

Output: cat c bat b hat h

4. Extracting & Splitting Methods

14 slice(start, end)

Extracts a section of a string. Supports negative indices (counted from the end).

```
str.slice(start, end)

let str = "JavaScript";
console.log(str.slice(0, 4));
console.log(str.slice(-6));

Output: Java Script
```

15 substring(start, end)

Similar to slice, but does not accept negative indices (they are treated as 0).

```
str.substring(start, end)

let str = "JavaScript";
console.log(str.substring(0, 4));
console.log(str.substring(4, 0)); // swaps args automatically

Output: Java Java
```

16 substr(start, length) — legacy

Extracts a substring starting at a given index for a given length. Deprecated; prefer slice().

```
str.substr(start, length)

let str = "JavaScript";
console.log(str.substr(4, 6));

Output: Script
```

17 split(separator, limit)

Splits a string into an array of substrings based on a separator.

```
str.split(separator, limit)

let str = "apple,banana,cherry";
console.log(str.split(","));
console.log("Hello".split(""));

Output: [ 'apple', 'banana', 'cherry' ] [ 'H', 'e', 'l', 'l', 'o' ]
```

5. Replacing & Case Methods

18 `replace(searchValue, newValue)`

Replaces the first match of a substring or regex with a new value.

```
str.replace(searchValue, newValue)
```

```
let str = "I like cats. Cats are great.";
console.log(str.replace("cats", "dogs"));
console.log(str.replace(/cats/gi, "dogs"));
```

Output: I like dogs. Cats are great. I like dogs. dogs are great.

19 `replaceAll(searchValue, newValue)`

Replaces all matches of a substring (or global regex) with a new value.

```
str.replaceAll(searchValue, newValue)
```

```
let str = "2024-01-01";
console.log(str.replaceAll("-", "/"));
```

Output: 2024/01/01

20 `toUpperCase()` / `toLowerCase()`

Converts a string to all uppercase or all lowercase characters.

```
str.toUpperCase() str.toLowerCase()
```

```
let str = "Hello World";
console.log(str.toUpperCase());
console.log(str.toLowerCase());
```

Output: HELLO WORLD hello world

6. Trimming, Padding & Repeating

21 trim() / trimStart() / trimEnd()

Removes whitespace from both ends, the start only, or the end only.

```
str.trim() str.trimStart() str.trimEnd()

let str = "  Hello World  ";
console.log("[ " + str.trim() + " ]");
console.log("[ " + str.trimStart() + " ]");
console.log("[ " + str.trimEnd() + " ]");

Output: [Hello World] [Hello World ] [ Hello World]
```

22 padStart(targetLength, padString)

Pads the current string from the start with another string until it reaches the target length.

```
str.padStart(targetLength, padString)

let str = "7";
console.log(str.padStart(3, "0"));
console.log("99".padStart(5, "ab"));

Output: 007 aba99
```

23 padEnd(targetLength, padString)

Pads the current string from the end with another string until it reaches the target length.

```
str.padEnd(targetLength, padString)

let str = "5";
console.log(str.padEnd(3, "0"));

Output: 500
```

24 repeat(count)

Returns a new string with the original string repeated the given number of times.

```
str.repeat(count)

let str = "ab";
console.log(str.repeat(3));

Output: ababab
```

7. Comparison, Encoding & Static Methods

25 normalize(form)

Returns the Unicode Normalization Form of a string, useful when comparing strings with accented characters.

```
str.normalize(form)

let str1 = "\u00F1";      // ñ (single code point)
let str2 = "n\u0303";     // n + combining tilde
console.log(str1 === str2);
console.log(str1.normalize() === str2.normalize());

Output: false true
```

26 localeCompare(compareString)

Compares two strings in the current locale, returning -1, 0, or 1.

```
str.localeCompare(compareString)

console.log("apple".localeCompare("banana"));
console.log("banana".localeCompare("apple"));
console.log("apple".localeCompare("apple"));

Output: -1 1 0
```

27 toString() / valueOf()

Returns the string value itself (mostly used internally by JS type coercion).

```
str.toString() str.valueOf()

let str = new String("Hello");
console.log(str.toString());
console.log(str.valueOf());
console.log(typeof str.valueOf());

Output: Hello Hello string
```

28 String.fromCharCode() / String.fromCodePoint()

Static methods that build a string from character codes / Unicode code points.

```
String.fromCharCode(...codes) String.fromCodePoint(...codes)

console.log(String.fromCharCode(72, 101, 108, 108, 111));
console.log(String.fromCodePoint(128512)); // 🍌

Output: Hello 🍌
```

29 String.raw()

A tag function for template literals that returns the raw string, ignoring escape sequences.

```
String.raw`template`

let path = String.raw`C:\Users\newFolder`;
console.log(path);

Output: C:\Users\newFolder
```

Quick Reference Table

Method	Purpose	Returns
charAt / at	Get character at index	string
charCodeAt / codePointAt	Get character code	number
concat	Join strings	string
includes / startsWith / endsWith	Substring check	boolean
indexOf / lastIndexOf	Find position	number
search / match / matchAll	Regex based search	number / array / iterator

slice / substring / substr	Extract part of string	string
split	String → array	array
replace / replaceAll	Replace text	string
toUpperCase / toLowerCase	Change case	string
trim / trimStart / trimEnd	Remove whitespace	string
padStart / padEnd	Add padding	string
repeat	Repeat string	string
normalize	Unicode normalization	string
localeCompare	Locale-aware comparison	number
String.fromCharCode / fromCodePoint	Build string from codes	string
String.raw	Raw template string	string

8. Interview Questions — Question / Input / Output

Common coding interview questions built around JavaScript string methods, each with the question, sample input, code, and expected output.

Q1. Reverse a string.

Input:

```
let str = "hello";
```

Code:

```
function reverseString(str) {
  return str.split("").reverse().join("");
}
console.log(reverseString(str));
```

Output: olleh

Q2. Check if a string is a palindrome.

Input:

```
let str = "madam";
```

Code:

```
function isPalindrome(str) {
  let clean = str.toLowerCase().replaceAll(" ", "");
  return clean === clean.split("").reverse().join("");
}
console.log(isPalindrome(str));
```

Output: true

Q3. Count the number of vowels in a string.

Input:

```
let str = "JavaScript Interview";
```

Code:

```
function countVowels(str) {
  let matches = str.match(/[aeiou]/gi);
  return matches ? matches.length : 0;
}
console.log(countVowels(str));
```

Output: 6

Q4. Find the first non-repeating character in a string.

Input:

```
let str = "swiss";
```

Code:

```
function firstNonRepeat(str) {
  for (let ch of str) {
    if (str.indexOf(ch) === str.lastIndexOf(ch)) return ch;
  }
  return null;
}
console.log(firstNonRepeat(str));
```

Output: w

Q5. Check if two strings are anagrams of each other.

Input:

```
let str1 = "listen";
let str2 = "silent";
```

Code:

```
function isAnagram(a, b) {
  const norm = s => s.toLowerCase().split("").sort().join("");
  return norm(a) === norm(b);
}
console.log(isAnagram(str1, str2));
```

Output: true

Q6. Capitalize the first letter of every word in a sentence.

Input:

```
let str = "javascript string methods guide";
```

Code:

```
function capitalizeWords(str) {
  return str
    .split(" ")
    .map(w => w.charAt(0).toUpperCase() + w.slice(1))
    .join(" ");
}
console.log(capitalizeWords(str));
```

Output: Javascript String Methods Guide

Q7. Count occurrences of each character in a string.

Input:

```
let str = "banana";
```

Code:

```
function charCount(str) {
  let result = {};
  for (let ch of str) {
    result[ch] = (result[ch] || 0) + 1;
  }
  return result;
}
console.log(charCount(str));
```

Output: { b: 1, a: 3, n: 2 }

Q8. Remove all whitespace from a string.

Input:

```
let str = " J a v a S c r i p t ";
```

Code:

```
function removeSpaces(str) {
  return str.replaceAll(" ", "");
}
console.log(removeSpaces(str));
```

Output: JavaScript

Q9. Check whether a string contains only digits.

Input:

```
let str1 = "12345";
let str2 = "123a5";
```

Code:

```
function isNumeric(str) {
  return /^d+$/ .test(str);
}
console.log(isNumeric(str1));
console.log(isNumeric(str2));
```

Output: true false

Q10. Truncate a string to a given length and append "...".

Input:

```
let str = "This is a long JavaScript sentence";
let limit = 15;
```

Code:

```
function truncate(str, limit) {
  return str.length > limit ? str.slice(0, limit) + "..." : str;
}
console.log(truncate(str, limit));
```

Output: This is a long...

Q11. Find the longest word in a sentence.

Input:

```
let str = "The quick brown fox jumped";
```

Code:

```
function longestWord(str) {  
  return str  
    .split(" ")  
    .reduce((a, b) => (b.length > a.length ? b : a));  
}  
console.log(longestWord(str));
```

Output: jumped

Q12. Convert a string to title case using padStart for a numbering prefix.

Input:

```
let items = ["milk", "eggs", "bread"];
```

Code:

```
items.forEach((item, i) => {  
  let num = String(i + 1).padStart(2, "0");  
  console.log(`${num}. ${item.charAt(0).toUpperCase() + item.slice(1)}`);  
});
```

Output: 01. Milk 02. Eggs 03. Bread

Q13. Check if a string starts and ends with the same character.

Input:

```
let str = "level";
```

Code:

```
function sameEnds(str) {  
  return str.at(0) === str.at(-1);  
}  
console.log(sameEnds(str));
```

Output: true

Q14. Extract all email addresses from a block of text.

Input:

```
let text = "Contact us at info@example.com or support@test.org";
```

Code:

```
function extractEmails(text) {  
  return text.match(/[\\w.-]+@[\\w.-]+\\.\\w+/g);  
}  
console.log(extractEmails(text));
```

Output: ['info@example.com', 'support@test.org']

Q15. Compare two strings ignoring case using localeCompare.

Input:

```
let str1 = "Apple";  
let str2 = "apple";
```

Code:

```
function equalsIgnoreCase(a, b) {  
  return a.localeCompare(b, undefined, { sensitivity: "base" }) === 0;  
}  
console.log(equalsIgnoreCase(str1, str2));
```

Output: true